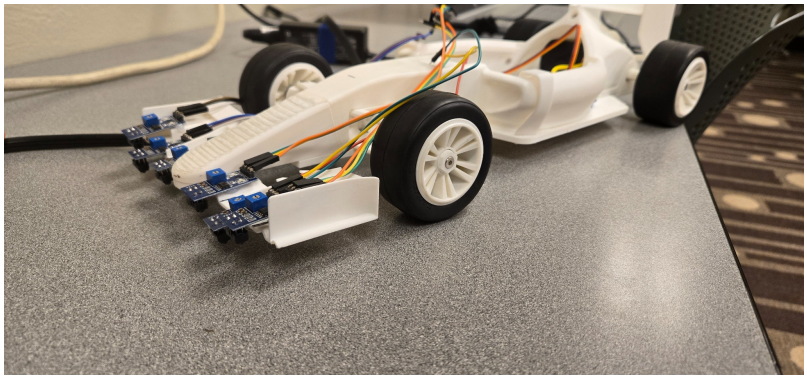
ENGINEERING CLUB COLLABORATION / PROJECT REPORT

DUAL MODE F1 RC CAR

Radio Control + Autonomous Driving
2015 and 2023 Vehicle Models



Prototype vehicle with the front-mounted IR sensor array. [I3]

Zarraf Alam Fahmid

PCB Designer

A collaborative project with the **Schoolcraft Engineering Club**. My individual contribution focused on **PCB design and manufacturing preparation**.

Vehicle development, mechanical design, firmware, and testing were team work.

Abstract

The Schoolcraft Engineering Club developed **2015 and 2023 F1-style RC models** and an autonomous control system combining IR sensing, PID steering, Pixy2 vision, and ESC control. My principal contribution was the custom Arduino Nano interface PCB. This report presents the vehicle architecture, component list, PCB design, recorded test results, and proposed improvements.

Documented item	Value and scope
Custom PCB	44.58 × 23.62 mm; two-layer design
2015 recorded speed	8.83 m/s = approximately 19.8 mph
2023 recorded speed	9.94 m/s = approximately 22.2 mph
Autonomous sensing	6 IR sensor inputs and Pixy2 color signatures

Recorded speeds are interval averages from the project measurement records, converted here to mph. [S1, S2]

Report guide

Vehicle models and system scope: page 2
Component list and equipment: pages 3-4
PCB design and manufacturing preparation: pages 5-6
Firmware and wiring: pages 7-8
Test results, photographs, and next revision: pages 9-14
Sources and technical references: pages 15-16
Complete autonomous source code: pages 17-22

Prepared 17 October 2026 | Updated 24 October 2026

1 Vehicle models and system scope

The labels **2015** and **2023** identify two vehicle models, not the project dates. Model-specific measurements below come from the team comparison sheet. The supplied autonomous firmware is a single configuration and does not identify which model or revision it ran on. [S1, S2, C1]

Recorded parameter	2015 model	2023 model
Total mass	0.847 kg	1.35 kg
Body dimensions L × W × H	427 × 170 × 102 mm	437 × 181 × 90.5 mm
Frontal area	0.0111 m²	0.0103 m²
Wheel radius	32 mm	32 mm
Listed gear ratio	2.86	2.86
Recorded speed	8.83 m/s	9.94 m/s

The dimension values are transcribed from the worksheet. The 2023 CAD screenshot separately displays a height of approximately 103 mm, so the 90.5 mm worksheet height should be reconciled against the final body configuration. [S2, I4]

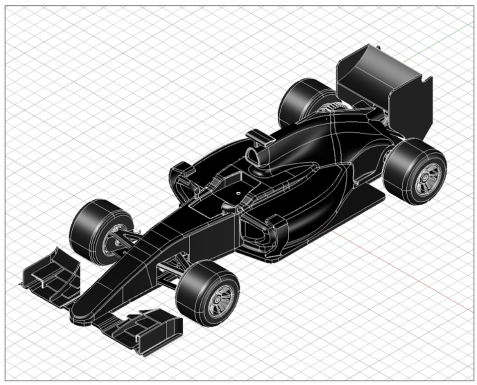


Figure 1a. Vehicle CAD reference; model not explicitly labeled. [I4]

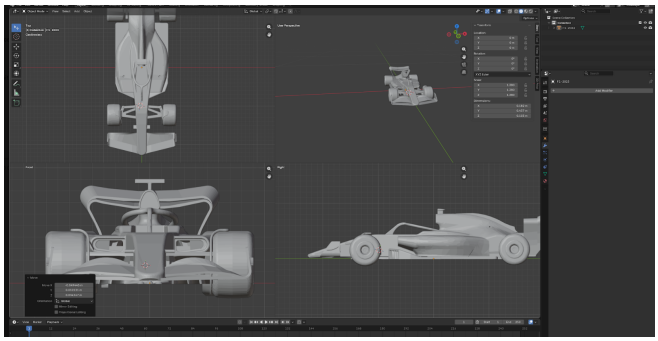


Figure 1b. CAD object labeled F1 2023. [I4]

System functions

The RC hardware provides a transmitter and receiver for manual operation. The autonomous controller reads a front IR array, calculates steering correction, checks trained Pixy2 red and green signatures, and commands a steering servo and ESC. The project files also describe off-board video analysis for vehicle speed measurement; that is a separate use of computer vision from the onboard Pixy2 traffic-light function. [S1, S4, C1]

The two modes are radio-controlled driving and autonomous line following. According to the author, the operating mode was changed manually; the Arduino did not select modes automatically. The exact manual changeover wiring or procedure is not documented. The supplied autonomous code contains no receiver-reading or mode-switching routine, and does not implement obstacle avoidance. [A1, C1]

2 Components and electrical interfaces

This list combines the schematic, firmware, photographs, and order register. Quantities refer to the documented autonomous configuration or the procurement record, as stated. The files do not provide a complete, separately allocated bill of materials for both the 2015 and 2023 models. [S1, S4, I1, I3, C1, A1]

Component and specification	Quantity or scope	Function and evidence
Arduino Nano ATmega328P-based controller	1 in the supplied autonomous configuration	Runs sensor processing, PID steering, Pixy2 decisions, and ESC commands. One vehicle-controller Nano is listed received. [C1, S4]
Pixy2 Smart Vision Sensor	1 listed received	Onboard red and green traffic-light signature detection. [C1, S4]
IR reflective sensor modules	6 used; 10-unit pack listed received	Front line-sensing array; six inputs are present in code and the physical prototype. Exact module part number not supplied. [C1, I3, S4]
Electronic speed controller	1 controlled by the firmware; 1 replacement listed received	Drive-motor control. ESC model, current rating, and braking configuration are not specified. [C1, S4]
Steering servo	1 controlled by the firmware	Actuates the steering linkage; servo and motor are included in the Phase 1 hardware-kit description. Exact servo model not supplied. [C1, S4]
Electric drive motor	Motor included in Phase 1 kit	Propulsion; the analysis lists 12,000 rpm no-load speed. Exact motor model and allocation to each vehicle are not identified. [S1, S4]
HRB LiPo battery 2S, 7.4 V, 2200 mAh, XT60	1 pack listed received	Vehicle energy source. These specifications come from the order description. [S4]
RC transmitter and receiver	1 set plus 1 replacement receiver listed received	Manual radio control; mode-switching wiring is not included in the supplied code. [S4, C1]
Custom Nano interface PCB 44.58 × 23.62 mm, two layers	1 board design documented	Connects controller, servo, ESC control, IR signals, and IR power. Physical assembly quantity is not established. [I1, A1]

PCB connector provisions

Connection	Quantity in the design	Purpose
Servo and ESC headers	2 connectors with 3 positions each	Ground, supply, and control signal.
IR signal and power headers	1 six-position and 1 two-position connector	Six IR signal connections and separate GND/VCC power connections.
Nano socket arrangement	2 rows of 15 positions	2.54 mm female-header connection planned for removal with power disconnected.

Connector counts follow the schematic and Nano footprint. The socketed assembly plan uses 2.54 mm headers; the final manufacturing BOM should identify the selected connector parts and populated quantities. [I1, A1]

2 Components and test equipment continued

Mechanical components and materials

Component group	Documented scope	Purpose and source
Printed chassis and body	Separate 2015 and 2023 vehicle geometries	Vehicle structure and packaging. Model-specific final material and printed-part counts are not fully allocated. [S2, S5, I3, I4]
Wheels and tires	4 wheels per illustrated vehicle; tire set listed received	Ground contact and traction; the print plan also includes mock-up tires. [I3, I4, S4, S5]
Steering linkage and mounts	Servo holder, servo-saver parts, push pins, hubs, and axle holders listed	Transfers servo motion to the front wheels and locates mechanical components. [S5]
Drive gears and axles	Pinion, spur gear, and axles listed; ratio 2.86 used in the analysis	Transmits motor torque to the wheels. Tooth counts are not established here. [S1, S2, S5]
Bearings and fasteners	1 Phase 1 kit and 1 Phase 2 kit listed received	Mechanical assembly. The Phase 2 kit explicitly excludes a motor and servo. [S4]
Print and support materials	PLA in the estimation sheet; PETG-CF, ABS, and ABS support material pending in the order register	Prototype fabrication planning. Final material use is not inferred from a pending order. [S4, S5]

Track hardware and measurement equipment

Item	Quantity or status	Role and source
Mini traffic-light LED modules	5-unit pack listed received	Trackside traffic-signal hardware. Installed count is not recorded. [S4]
ELEGOO Nano boards	3-unit pack listed received	Controllers ordered for the traffic lights, separate from the vehicle-controller Nano. [S4]
Transparent RGB LEDs	100-unit pack listed received	Traffic-light construction; installed LED count is not recorded. [S4]
Track marking materials	Black gaffer tape and matte black paint listed pending	Low-reflection track preparation; photographs document the test path and signal props. [S4, I5]
Camera and computer	Used in the documented speed-analysis method	Off-board video timing and Python/OpenCV processing; equipment models not supplied. [S1]
Digital multimeter and calipers	Listed as equipment to borrow	Electrical checks and dimensional measurements. [S4]
Pitot-tube equipment and Bluetooth interface	Pitot equipment listed received; Bluetooth interface listed borrowed	Supplementary airspeed and data-analysis work; completed integration is not established. [S4]

3 PCB Design

As the PCB designer for this **Schoolcraft Engineering Club collaboration**, I developed the Arduino Nano interface board in EasyEDA, from schematic capture and component placement to routing and manufacturing preparation. The board connects the controller to the steering servo, ESC control signal, six IR sensor signals, and IR power supply. [A1, I1, I2]

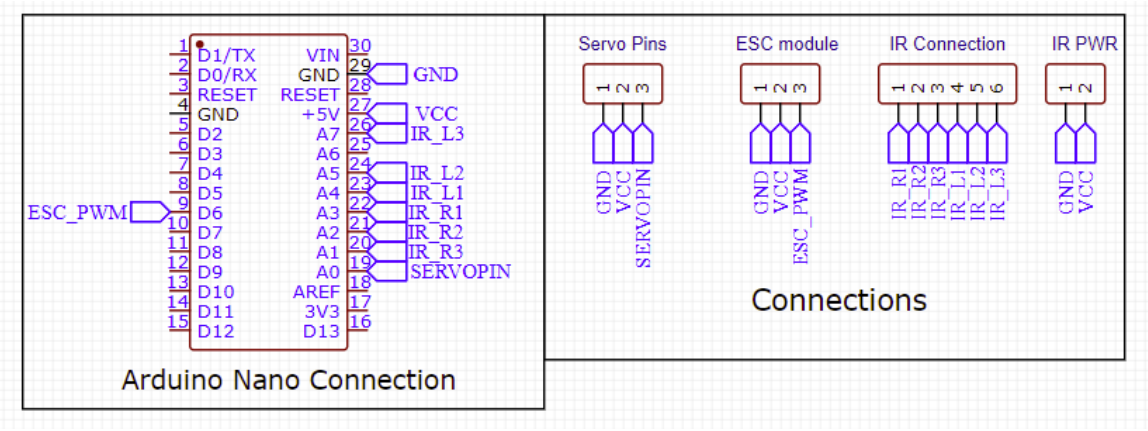


Figure 2. Nano and peripheral schematic. Signal assignments must match the firmware revision. [I1]

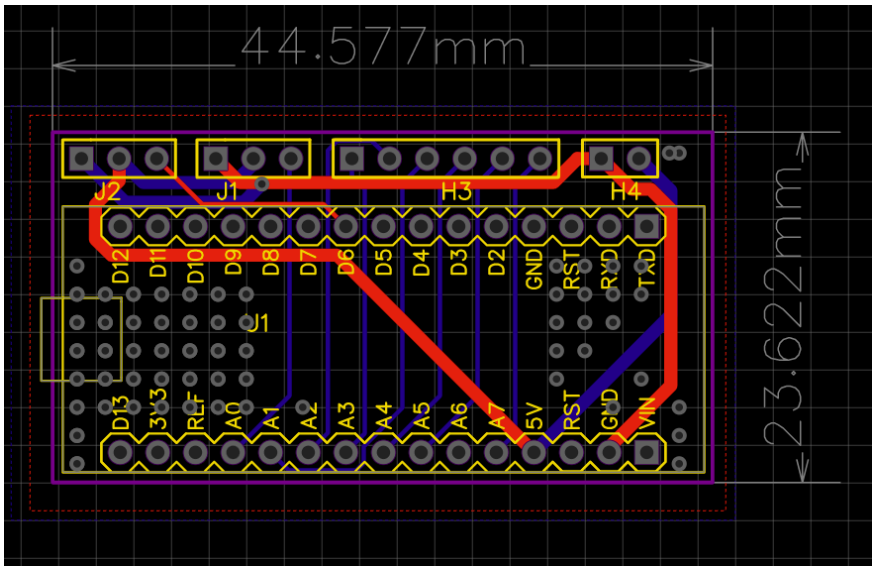


Figure 3. Routed board measuring approximately 44.577 × 23.622 mm, rounded to 44.58 × 23.62 mm. [I1]

Design constraints and decisions

I designed the **44.58 × 23.62 mm, two-layer FR-4 PCB** to fit the chassis, grouped the peripheral connectors for access, and specified **35 mil power traces with 1 oz copper**. The design specifications should accompany the native PCB files and fabrication order. [A1, I1]

4 Grounding and manufacturing preparation

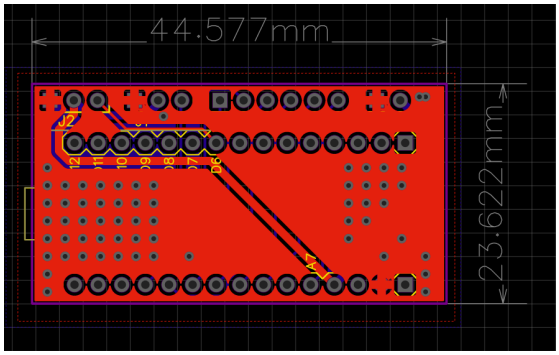


Figure 4a. Filled copper and via array. [I1]

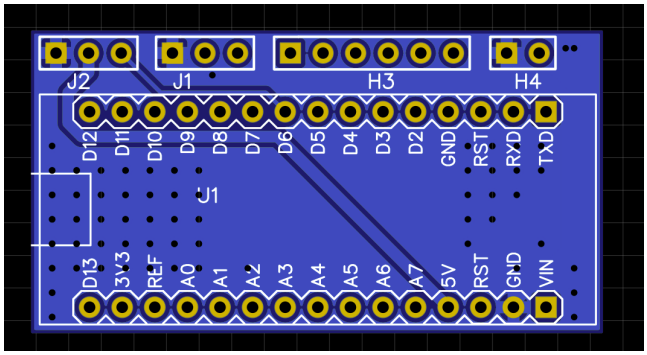


Figure 4b. Opposite-side copper; via GND connection confirmed by the author. [I1, A1]

Ground stitching and thermal intent

I connected the top and bottom GND pours with **ground stitching vias** to support ground continuity and heat spreading through the copper. Their thermal purpose is heat conduction between layers; any temperature reduction would require measurement under load. [I1, A1, R3]

Assembly and serviceability

I planned a removable Nano connection using **2.54 mm female headers** for replacement with power disconnected. Manufacturing preparation included a **1 × 5 panel**, **5 mm handling rails**, manufacturer-added tooling holes, and top-side assembly settings. The Nano was excluded from factory placement; the headers require their own BOM entries or manual installation. [A1, R2]

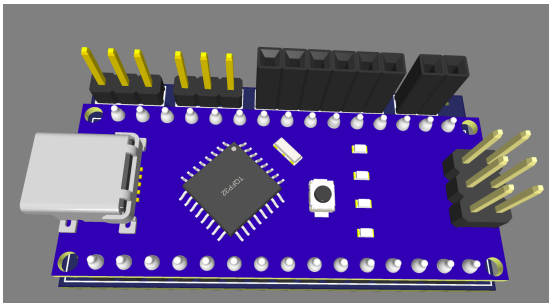


Figure 5. Nano and interface-board CAD assembly preview. [I2]

Fabrication verification should compare the final BOM/CPL, approved placement preview, and assembled board against the design.

5 Autonomous firmware implementation

The project’s Arduino firmware integrates sensing, steering, traffic-light recognition, and motor commands. The firmware is presented as team work, while my contribution focused on the controller interface PCB. Full source listings appear in Appendices A to C. [C1-C3]

Six sensor processing

Each analog reading is converted by `map(reading, 0, 1024, 0, 2)`. With integer arithmetic and an input range of 0 to 1023, the result is 0 below 512 and 1 from 512 upward. The custom library selects contributions based on the number of active readings on each side; it is not a conventional weighted-average position calculation. [C1-C3, R1]

Array position	L3	L2	L1	R1	R2	R3
Weight	+7	+5	+2	-2	-5	-7

The header comments specify nominal sensor offsets of 2, 5, and 7 cm from the centerline. Their units explain the weight values but do not establish calibrated physical position accuracy. For two active sensors on a side, the library selects the outer active sensor. Three active sensors on a side contribute zero. [C2, C3]

PID steering

The steering correction combines proportional error, a time-integrated error term, and an exponentially smoothed derivative. The integral is clamped to ±10. Servo commands are centered at 90° and constrained to 20°-160°; these are servo command values, not measured road-wheel angles. [C1]

Setting	Implemented value
PID gains	Kp 12.0; Ki 0.1; Kd 0.8
Derivative smoothing	D = 0.8 × previous D + 0.2 × raw D
Startup	1500 µs neutral command for 5 seconds
Forward routine	1370 µs for 25 ms, then 1500 µs for 25 ms

Pixy2 traffic light decisions

The program requests color-connected-component blocks and returns on the first red or green signature: red is signature 1 and green is signature 3. No matching signature permits movement. Pixy2 normally orders these blocks by area, so the code does not give red unconditional priority when both appear. The software commands neutral throttle on a stop decision; stopping distance and braking behavior were not measured. [C1, R4]

6 Wiring revisions and subsystem checks

The schematic and the integrated firmware differ at three signal assignments. This may reflect different revisions or wiring arrangements, but the exact hardware and firmware pairing cannot be established from the project records alone. [I1, C1]

Function	PCB schematic	Integrated code	Status
Steering servo	A0	A0	Matches
ESC signal	D6	A4	Revision mismatch
Left IR 1	A4	A5	Revision mismatch
Left IR 2	A5	A6	Revision mismatch
Left IR 3	A7	A7	Matches
Right IR 1 / 2 / 3	A3 / A2 / A1	A3 / A2 / A1	Matches

A4 is assigned to an IR input in the schematic but to the ESC output in the code. Do not describe this source set as an exact reproducible wiring and firmware combination until the final pin map is reconciled. The Pixy2 cable, interface configuration, and power connection also need to be documented separately.

Supplied standalone test sketches

Sketch	Programmed test behavior
ESC	1400 µs forward for 3 s; 1500 µs neutral for 3 s, following a 5 s startup delay.
IR array and steering	Reads six sensors and runs PID steering; it does not command the drive motor.
Pixy2	Prints red or green detections to the serial monitor; it does not actuate the vehicle.
Servo	Uses pin 3 and alternates 0° and 180° commands every 3 s; these differ from the integrated operating limits.
Traffic light	Separate red and green LEDs: green for 5 s and red for 7 s.

These sketches document how the subsystems could be exercised independently. No pass counts, recorded test log, detection-distance series, or repeated stopping trials were supplied, so the sketches are not treated as proof of a success percentage. [C4]

7 Recorded speed results by model

The team recorded vehicle speed using a 2.65 m distance between parking-space lines and video-based timing. The findings describe motion segmentation, contour detection, bounding-box tracking, and line-crossing events using Python and OpenCV. This measurement system was off the vehicle. The video files and Python source were not supplied for independent replay. [S1 pp 3-6, S2]

Quantity	2015 model	2023 model
Distance	2.65 m	2.65 m
Listed elapsed time	0.300 s	0.267 s
Recorded speed	8.83 m/s	9.94 m/s
Converted speed	31.8 km/h / 19.8 mph	35.8 km/h / 22.2 mph

The recorded 2023 value is approximately **12.6% higher** than the recorded 2015 value: (9.94 - 8.83) / 8.83 × 100. This compares the two recorded results; it does not establish a repeatable improvement caused by the PCB, body design, or control algorithm. Each model has one summary result in the supplied table. [S1, S2; calculation in this report]

Calculation and precision

For two line-crossing frame indices, use the following calculation. The source findings contain a typographical error showing a ratio of frame numbers; elapsed frames must be their difference.

```
elapsed frames = end frame - start frame
elapsed time = elapsed frames / frames per second
interval average speed = 2.65 m / elapsed time
```

For the 2015 model, 2.65 / 0.300 = 8.833... m/s, consistent with 8.83 m/s. For the 2023 model, 2.65 / 0.267 = 9.925... m/s; the listed 9.94 m/s can be consistent with a more precise, unrounded time near 0.2666 s, but raw frame indices are needed to confirm it. The findings mention both 30 and 60 fps without assigning a frame rate to each result. [S1]

Measured values and estimates

The analytical model lists a gear-limited estimate of **14.1 m/s**, approximately **31.5 mph**. The recorded interval-average speeds are 19.8 mph and 22.2 mph. These quantities describe different conditions: a theoretical limit and observed performance over the timing interval. Separate timing records are needed to establish autonomous-mode speed. [S1, S2]

The findings' force table is internally inconsistent: its listed 81.7 N traction force, drag, and rolling resistance do not sum to its approximately 6.98 N net force. Predicted acceleration and terminal-speed claims need recalculation before being presented as validated results. [S1 pp 1-3]

7 Acceleration graph and speed analysis

The figures below reproduce the project's analytical acceleration plot and video-based speed measurement. Original images embedded in Findings Summary are used to retain their axis labels, timing lines, and tracking annotations. The analytical curve and the video measurement represent different forms of evidence. [S1]

2015 analytical acceleration curve



Figure 6. Acceleration versus velocity curve for the 2015 model, reproduced from the original project findings. This is an analytical plot; the force-model discrepancies discussed in Section 7 require reconciliation before treating its predictions as validated. [S1]

Video based speed measurement

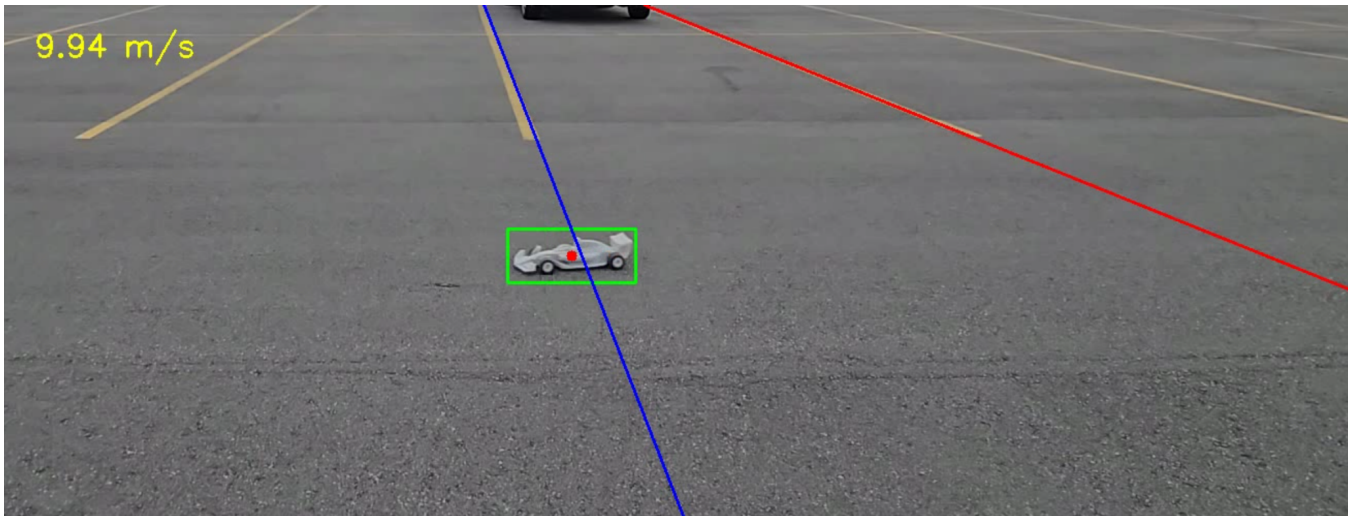


Figure 7. Video-analysis frame displaying 9.94 m/s. The green bounding box and red point identify the tracked vehicle; the red and blue lines define the timing interval. The recorded value is an interval-average speed. [S1, I6]

8 Turning geometry and track allowance

The steering study describes measuring wheel angles, averaging three angle measurements, and comparing a theoretical turning radius with three experimental semicircle traces. It treats left and right turns separately to account for steering asymmetry. Raw repetitions are not included, and the document does not explicitly assign the study to the 2015 or 2023 model. Its results are therefore kept in a separate turning-study table. [S3]

Recorded quantity	Left turn	Right turn
Wheelbase	26.3 cm	26.3 cm
Inner wheel angle	30.9°	38.7°
Outer wheel angle	20.4°	22.1°
Radius column 1	63.4 cm	56.1 cm
Radius column 2	60.7 cm	52.6 cm
Listed percent difference	4.45%	6.39%
Listed worst radius plus 10%	74.0 cm	64.3 cm

What the 10 percent means

The documented method adds a **10% design allowance** to the conservative turning radius when laying out the track. This is an engineering margin; it is not a measured 10% uncertainty, a steering accuracy figure, or a probability of successful cornering. The basic design rule is $R_{track} = 1.10 \times R_{selected}$. [S3 p 1]

What remains unresolved

The source prose distinguishes theoretical and experimental radii, but the table labels its two radius columns “Experimental” and “Actual.” This report retains neutral column labels until that mapping is confirmed. Applying 10% to the larger displayed radius gives **69.74 cm left** and **61.71 cm right**, rather than the listed 74.0 and 64.3 cm. The listed design radii may use undisclosed worst-case repetitions; the raw records are needed to reproduce them. [S3 p 3; arithmetic checked here]

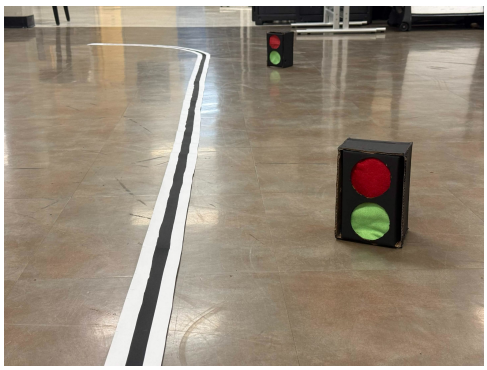


Figure 8. Track and traffic-light props. This photograph does not establish a measured corner radius or successful-run percentage. [I5]

9 Prototype fabrication and integration

The physical photographs show a white printed chassis with six IR modules distributed across the front. A close-up records sensor positioning over a black test strip. The CAD and slicer images document body geometry and packaging work at team level; their authorship and model assignments are not all labeled. [I3-I5]

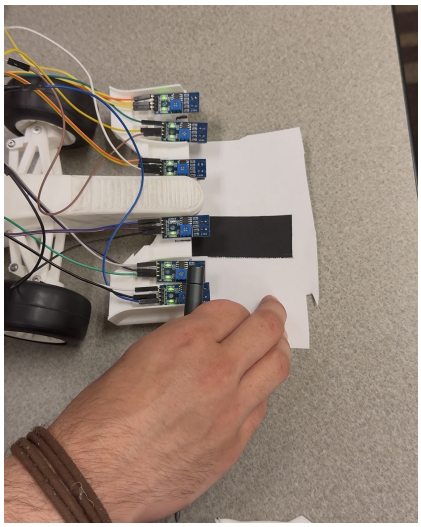


Figure 9a. IR array positioned over a black reference strip. [I5]

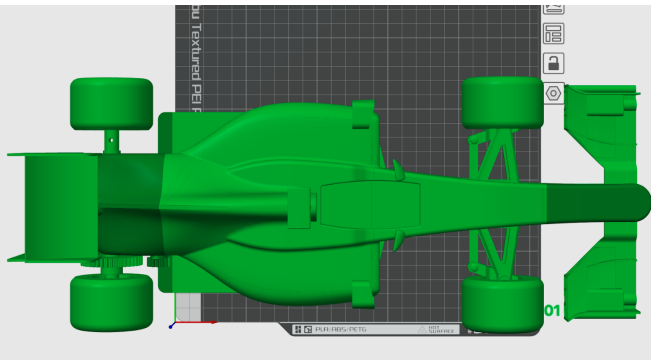


Figure 9b. Supplied body and wheel geometry in a slicer view. [I4]

Phase 1 print planning

The filament worksheet estimates the first seven numbered batches for a Bambu P1S with a 0.4 mm nozzle. It lists PLA and includes mock-up tires; these entries are planning estimates, not evidence of the final material or actual consumption of both models. An eighth replacement-parts batch has no complete totals and is excluded from the figures below. [S5]

Planning item	Listed total
Model volume	411.45 cm³
Part filament mass	510.20 g
Support mass	65.17 g
Part plus support mass	575.37 g, calculated from listed totals
Estimated printing time	22 h 38 min

The worksheet includes components such as the chassis, body, servo holder, steering parts, gears, wheels, and wings. The order register separately records a 7.4 V 2S 2200 mAh battery, Pixy2 camera, Nano, IR sensors, and RC hardware as received. Receipt records support component availability, not measured electrical performance or installation in a specific vehicle. [S4, S5]

The cost register combines received, borrowed, and pending items and contains more than one displayed total. A final project cost requires reconciliation of those entries and any outstanding purchases. [S4]

9 Vehicle photographs

Additional views document the printed vehicle body and the assembled test configuration on the marked path. These captures show hardware arrangement and test context; the individual photographs do not explicitly identify a model or a measured performance result. [I6]

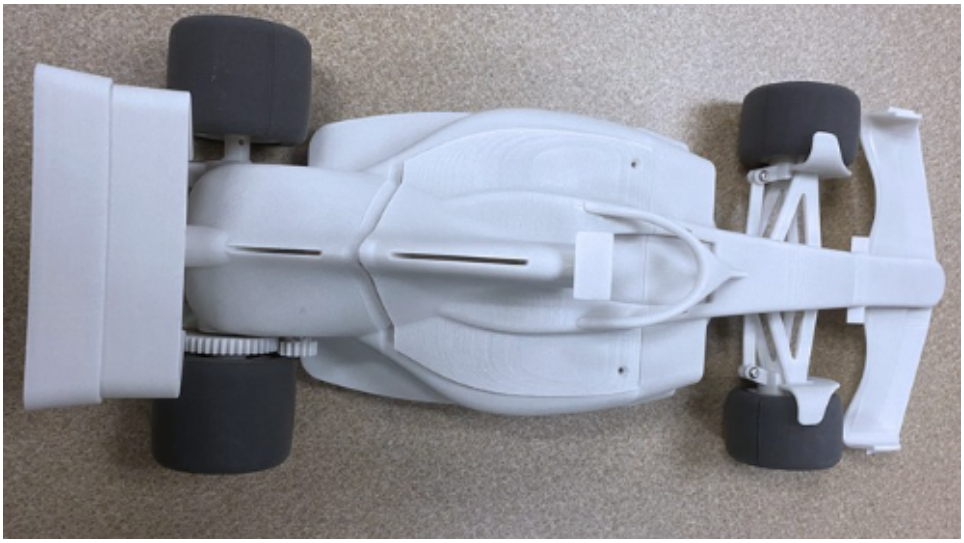


Figure 10a. Top view of the printed F1-style body, wheel assemblies, and exposed drive gearing. [I6]

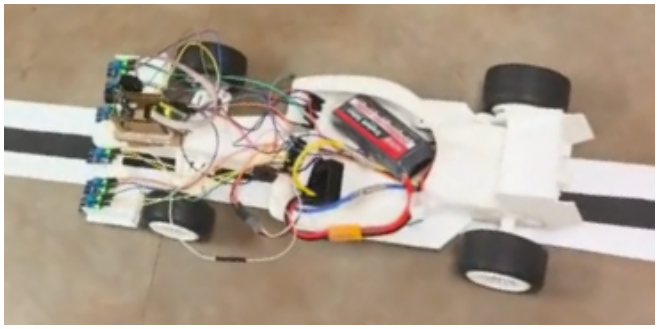


Figure 10b. Vehicle electronics and front IR array positioned over the marked test path. [I6]

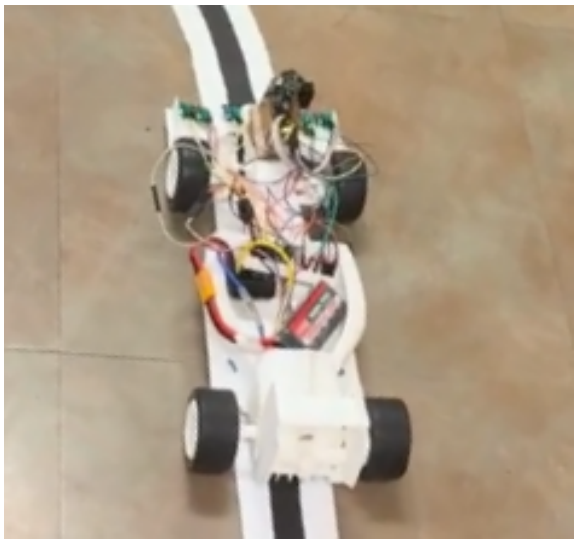


Figure 10c. Additional view of the assembled vehicle on the test path. [I6]

The original image files are embedded without additional lossy compression. Close inspection remains limited by the detail captured in the source screenshots.

10 Verification limits and next revision

The project records document a physical prototype, autonomous firmware, PCB design, and comparative vehicle measurements. Review of the wiring, source code, and calculations identified the following priorities for the next revision.

Team-reported development challenges

Pixy2 recognition required repeated changes to the traffic-light targets' color and shape, along with signature retraining before use. The team also reported difficulty finding reliable printing services, fit problems, broken parts, and reprints or rebuilds. Procurement delays prompted design changes, while assembly and vehicle testing competed with autonomous-function development for time. These retrospective team accounts describe the development process rather than measured detection accuracy or verified fault causes. [T1]

Finding	Next action
PCB and firmware pin mismatch	Record the final wiring map and use matching pin definitions before uploading firmware.
Ambiguous IR patterns	Define expected track polarity and lost-line/intersection behavior. All-zero, all-one, and three-active-on-one-side patterns currently can yield zero error.
Traffic-light precedence	Scan all relevant blocks and define explicit red priority, no-detection behavior, and camera-error handling.
PID timing across stops	Reinitialize or deliberately manage elapsed time and controller state when starting or resuming.
Blocking throttle routine	Measure the actual control-loop interval. The two 25 ms delays alone add 50 ms, so the full update rate is below 20 Hz when the forward routine runs.
Thermal and power claims	Measure loaded rail voltage, current, trace temperature, and connector temperature before asserting performance gains.
Vehicle speed and turning results	Retain model, mode, frame rate, distance, repeated trials, and raw turning data with each result.

Scope of the code checks

Logical review of all 64 binary IR patterns shows that the validator accepts 54 patterns and rejects 10. These counts describe the software's acceptance behavior, rather than vehicle tracking accuracy. The appendices retain the prototype logic and the issues described above. Compilation and hardware testing should follow the wiring and control revisions.

Contribution statement

My contribution centered on the Nano interface PCB: translating peripheral connections into a compact two-layer layout, adding GND stitching, and addressing assembly and maintenance considerations. The firmware, vehicle CAD, fabrication planning, and speed and turning analyses form part of the team project.

11 Source register and reproducibility

Bracketed identifiers identify the project records and technical references used throughout this report. Numerical discrepancies and limits of the measurements are discussed in the relevant sections.

ID	Source and use
S1	Findings Summary .pdf, 6 pages. Speed method, motor and vehicle parameters, recorded timings, and force-analysis review.
S2	Velocity & Speed Data - Sheet1.pdf, 1 page. Separate 2015 and 2023 measurements and theoretical estimates.
S3	Ackermann Geometry.pdf, 4 pages. Steering method, radius table, and 10% track allowance.
S4	Order Requests - Winter 2026.pdf, 1 page. Component availability and procurement context.
S5	Filament Estimation - Phase 1 Car.pdf, 1 page. Print-planning estimates.
I1	Screenshot_2026-06-02_102901, 102921, 102930, and 102954.png; image(20261017-222120).png. Schematic, routed and filled layouts, and board dimensions.
I2	Screenshot_2026-06-02_103012.png and 103025.png. PCB assembly and board renders.
I3	20260607_153318.jpg and Screenshot 2026-10-17 172242.png. Physical prototype; overlapping views.
I4	Screenshots 2026-10-17 172231, 172237, 172251, and 172301.png. Slicer and CAD references; 172301 includes the F1 2023 object label.
I5	IMG_3805.jpg and IMG_3807.png. Track props and IR-array setup.
I6	Vehicle and test screenshots dated 23 October 2026: 122502, 123128, 123210, and 123350. The acceleration plot and speed-analysis frame use original embedded images from S1.
C1-C3	Project firmware: the integrated Arduino program, IrSensorArray.hpp, and IrSensorArray.cpp. Full listings appear in Appendices A-C.
C4	Five subsystem test sketches covering the ESC, IR sensors, Pixy2, steering servo, and traffic-light LEDs. See Section 6.
A1	Author’s technical account of PCB responsibility, manufacturing preparation, GND connections, camera setup, and manual mode changeover.
T1	Retrospective Engineering Club feedback relayed by the author. Qualitative accounts of Pixy2 target changes and retraining, printing and fit issues, rebuilds, procurement delays, and test scheduling.

Software dependencies: Arduino Servo library, Pixy2 library, and the custom IrSensorArray source. Arduino IDE/core and dependency versions were not supplied. To reproduce the build, retain the final native EasyEDA files, Gerber/drill files, BOM/CPL, wiring revision, Pixy signature settings, and build environment alongside the source.

12 Technical references and appendix guide

Primary technical references

R1 Arduino map reference source. Integer mapping behavior. Arduino reference repository, Language/Functions/Math/map.adoc.
github.com/arduino/reference-en/blob/master/Language/Functions/Math/map.adoc

R2 JLCPCB Component Matching Guidelines for PCBA Orders. Selected components and Do Not Place behavior.
jlcpcb.com/help/article/component-matching-guidelines-for-pcba-orders

R3 Texas Instruments PowerPAD Layout Guidelines SLOA120. Thermal-via heat paths depend on connections to device thermal copper and planes.
ti.com/lit/an/sloa120/sloa120.pdf

R4 Pixy2 Color Connected Components API. getBlocks, signature identifiers, ordering by block area, and error returns.
docs.pixycam.com/wiki/doku.php?id=wiki:v2:ccc_api

References reviewed 17 October 2026. External documentation explains component or API behavior; it does not verify the project's measured results.

Appendix contents

Appendix	Listing
A	AutonomousCar.ino - integrated sensing, steering, camera, and ESC program
B	IrSensorArray.hpp - class declaration and sensor weights
C	IrSensorArray.cpp - sensor validation and error-selection logic

The integrated program is listed as AutonomousCar.ino, followed by its custom sensor-library header and implementation. The listings preserve the prototype logic, with consistent whitespace and visual wrapping of long lines. Continuation lines carry no source line number.

To compile a sketch, place AutonomousCar.ino in a matching AutonomousCar folder and make the two custom library files available through the Arduino library structure or an appropriate local include arrangement. Install Servo and Pixy2 dependencies. The original angle-bracket include and pin assignments are preserved in Appendix A; resolve the documented wiring mismatch before operating hardware.

Further validation

The next test campaign should measure traffic-light detection reliability, lap completion, steering repeatability, PCB temperature, and autonomous-mode speed. Board assembly inspection and comparisons across controlled hardware revisions would support a more complete evaluation of the PCB's contribution to system performance.

Appendix A AutonomousCar.ino

AutonomousCar.ino | supplied prototype source | listing 1 of 3

```
// ***********************
// INCLUDE Pixy2 and IrSensorArray Libraries !!!
// ***********************

#include <Pixy2.h>
#include <Servo.h>
#include <IrSensorArray.hpp>

#define FWD_SPEED  1370
#define NEUTRAL_SPEED 1500
#define REV_SPEED 1600

#define ESC_DELAY 25

#define MIN_PULSE_W 1000
#define MAX_PULSE_W 2000

#define STARTUP_DELAY 5000 // 5 seconds

#define RED 1
#define GREEN 3

// Servo PIN
#define SERVO_PIN A0
#define MOTOR_PIN A4

// IR Sensors
#define IR_L_1 A5
#define IR_L_2 A6
#define IR_L_3 A7

#define IR_R_1 A3
#define IR_R_2 A2
#define IR_R_3 A1

// Servo range: 20 - 160
#define SERVO_STRAIGHT 90
#define SERVO_LEFT 160
#define SERVO_RIGHT 20

#define MAX_ERROR 5.72

// PID
#define Kp 12.0
#define Ki 0.1
#define Kd 0.8

Pixy2 pixy;

Servo servoMotor;
Servo ESC;

float oldD = 0.0;
float prevError = 0.0;
float integral = 0.0;
unsigned long prevTime = 0;

int normalizeIrReading(int);
```

Appendix A continued

AutonomousCar.ino | supplied prototype source | listing 2 of 3

```
59  void printValues(int*, int);
60  bool trafficLightCheck();
61  void fwd();
62  void stop();
63
64
65  void setup() {
66    Serial.begin(9600);
67
68    servoMotor.attach(SERVO_PIN);
69
70    pinMode(IR_L_1, INPUT);
71    pinMode(IR_L_2, INPUT);
72    pinMode(IR_L_3, INPUT);
73
74    pinMode(IR_R_1, INPUT);
75    pinMode(IR_R_2, INPUT);
76    pinMode(IR_R_3, INPUT);
77
78    pixy.init();
79
80    ESC.attach(MOTOR_PIN, MIN_PULSE_W, MAX_PULSE_W);
81
82    ESC.writeMicroseconds(NEUTRAL_SPEED);
83    delay(STARTUP_DELAY);
84  }
85
86
87  void loop() {
88    const int sensL1 = normalizeIrReading(analogRead(IR_L_1));
89    const int sensL2 = normalizeIrReading(analogRead(IR_L_2));
90    const int sensL3 = normalizeIrReading(analogRead(IR_L_3));
91    const int sensR1 = normalizeIrReading(analogRead(IR_R_1));
92    const int sensR2 = normalizeIrReading(analogRead(IR_R_2));
93    const int sensR3 = normalizeIrReading(analogRead(IR_R_3));
94
95    const int irSensorArrayReadings[] = {sensL3, sensL2, sensL1, sensR1, sensR2, sensR3};
96
97    IrSensorArray irSensorArray(irSensorArrayReadings);
98
99    printValues(irSensorArray.getValues(), irSensorArray._NUM_SENSORS);
100
101   if (trafficLightCheck() && irSensorArray.isValid()) {
102     const float error = irSensorArray.sumWeights();
103
104     unsigned long now = millis();
105     float dt = (now - prevTime) / 1000.0;
106     if (dt <= 0.0) dt = 0.001;
107
108     const float P = error;
109     integral += error * dt;
110     const float rawD = (error - prevError) / dt;
111     const float D = 0.8 * oldD + 0.2 * rawD;
112
113     integral = constrain(integral, -10.0, 10.0);
114
115     const float correction = Kp * P+  Kd * D + Ki * integral;
116     int servoAngle = SERVO_STRAIGHT + (int)correction;
```

Appendix A continued

AutonomousCar.ino | supplied prototype source | listing 3 of 3

```
117      servoAngle = constrain(servoAngle, SERVO_RIGHT, SERVO_LEFT);
118
119      Serial.println(servoAngle);
120
121      servoMotor.write(servoAngle);
122
123      fwd();
124
125      oldD = D;
126      prevError = error;
127      prevTime = now;
128    } else stop();
129  }
130
131  int normalizeIrReading(int reading) {
132    return map(reading, 0, 1024, 0, 2);
133  }
134
135  void printValues(int* values, int size) {
136    for (int i = 0; i < size; i++) {
137      Serial.print(values[i]);
138      Serial.print(" ");
139    }
140    Serial.println();
141  }
142
143  bool trafficLightCheck() {
144    pixy.ccc.getBlocks();
145
146    for (int i = 0; i < pixy.ccc.numBlocks; i++) {
147      switch (pixy.ccc.blocks[i].m_signature) {
148        case GREEN:
149          return true;
150        case RED:
151          return false;
152      }
153   }
154
155   return true;
156  }
157
158  void fwd() {
159    ESC.writeMicroseconds(FWD_SPEED);
160    delay(ESC_DELAY);
161    ESC.writeMicroseconds(NEUTRAL_SPEED);
162    delay(ESC_DELAY);
163  }
164
165  void stop() {
166    servoMotor.write(SERVO_STRAIGHT);
167    ESC.writeMicroseconds(NEUTRAL_SPEED);
168    Serial.println("STOP");
169  }
```

Appendix B IrSensorArray.hpp

IrSensorArray.hpp | supplied prototype source | listing 1 of 1

```
#ifndef IRSENSORARRAY_HPP
#define IRSENSORARRAY_HPP

class IrSensorArray {
public:
    static constexpr unsigned int _NUM_SENSORS = 6;

    IrSensorArray(const int sensValues[]);
    bool isValid();
    float sumWeights();
    int* getValues();

private:

    static constexpr float _DIST = 4.0f / 2; // Distance between middle sensors is 4cm, so
from middle to each sensor is 2cm
    static constexpr float _DIST_2 = 3.0f + _DIST; // Distance between adjacent sensors is 3cm
    static constexpr float _DIST_3 = 2.0f + _DIST_2; // Distance between adjacent sensors is
2cm

    const float _WEIGHTS[_NUM_SENSORS / 2] = {-_DIST, -_DIST_2, -_DIST_3};

    int _sensValues[_NUM_SENSORS];

    float getWeight(char, int);
    float sumSide(char, unsigned int, unsigned  int);
};

#endif
```

Appendix C IrSensorArray.cpp

IrSensorArray.cpp | supplied prototype source | listing 1 of 2

```
#include "IrSensorArray.hpp"

IrSensorArray::IrSensorArray(const int sensValues[]) {
    for (int i = 0; i < this->_NUM_SENSORS; i++) {
        this->_sensValues[i] = sensValues[i];
    }
}

bool IrSensorArray::isValid() {
    int count = 0;

    for (int i = 0; i < _NUM_SENSORS; i++) {
        if (_sensValues[i] == 1) count ++;
    }

    for (int i = 0; i < _NUM_SENSORS; i++) {
        if (_sensValues[i] == 1 && count == 2) {
        if (i == 0) return _sensValues[i + 1] == 1;
        else if (i == _NUM_SENSORS - 1) return _sensValues[i - 1] == 1;

        return _sensValues[i + 1] == 1;
        }
    }

    return true;
}

float IrSensorArray::sumWeights() {
    return sumSide('L', 0, _NUM_SENSORS / 2) + sumSide('R', _NUM_SENSORS / 2, _NUM_SENSORS);
}

float IrSensorArray::getWeight(char side, int index){
    if (side == 'R' && index >= _NUM_SENSORS / 2 && index < _NUM_SENSORS) return
_WEIGHTS[index - 3];
    else if (side == 'L' && index >= 0 && index < _NUM_SENSORS / 2) return -_WEIGHTS[2 -
index];
    return 0.0;
}

float IrSensorArray::sumSide(char side, unsigned int start, unsigned int end) {
    if (side != 'L' && side != 'R') return 0.0;
    if (start < 0 || start >= _NUM_SENSORS) return -1.0;
    if (end <= 0 || end > _NUM_SENSORS || end <= start) return -1.0;

    const int k = side == 'R'? 1 : -1;

    int count = 0;

    for (int i = start; i < end; i++) {
        if (_sensValues[i] != 0) count += 1;
    }

    if (count == 1) {
        for (int i = start; i < end; i++) {
            if (_sensValues[i] != 0) return _sensValues[i] * getWeight(side, i);
        }
    } else if (count == 2) {
        if (side == 'R')
```

Appendix C continued

IrSensorArray.cpp | supplied prototype source | listing 2 of 2

```
            for (int i = end - 1; i >= start; i--) {
                if (_sensValues[i] != 0) return _sensValues[i] * getWeight(side, i);
            }
        else
            for (int i = start; i < end; i++) {
                if (_sensValues[i] != 0) return _sensValues[i] * getWeight(side, i);
            }
    }

    return 0.0;
}

int* IrSensorArray::getValues() {
    return this->_sensValues;
}
```